



Vietnam National University
Ho Chi Minh City University of Technology

Assignment 2

Phân loại ảnh với các mô hình học sâu cơ bản

Group 9

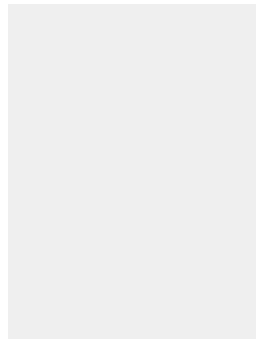
Course: **Học sâu và ứng dụng trong thị giác máy tính**

Lecturer: TS Lê Thành Sách

Members

Huỳnh Văn Tâm Thiện - 2470094

Trần Lê Phú - 2470307



Contents

- 01** Tập dữ liệu & EDA
- 02** Xây dựng các mô hình phân loại
- 03** Huấn luyện, đánh giá & so sánh
- 04** Tự hiện thực TransformerEncoder & ViT
- 05** Các kiến trúc kết hợp & cách embed ảnh khác nhau
- 06** Mô hình phân loại dựa trên LSTM/GRU
- 07** Tổng hợp kết quả

01

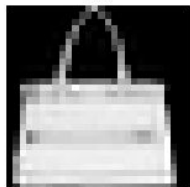
Tập dữ liệu & EDA

Tổng quan tập dữ liệu

- **Nguồn:** Fashion-MNIST
- **Tổng số mẫu:** 70.000 hình ảnh
- **Loại bài toán:** Phân loại ảnh.
- **Số lượng nhãn:** 10 lớp.
- **Chủ đề:** Ảnh về chủ đề thời trang.
- **Phân phối:** Tập train: 54.000 ảnh | Tập validation: 6.000 ảnh | Tập test: 10.000 ảnh.

Một số mẫu từ FashionMNIST

Bag



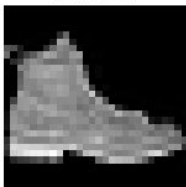
T-shirt/top



Coat



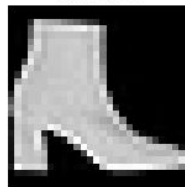
Ankle boot



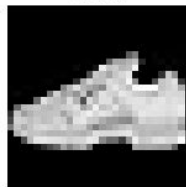
T-shirt/top



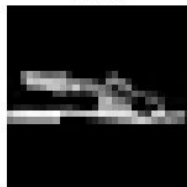
Ankle boot



Sneaker



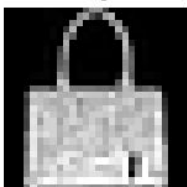
Sandal



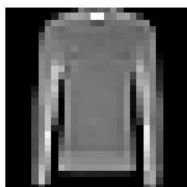
Shirt



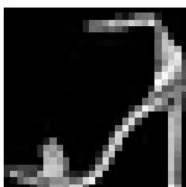
Bag



Pullover



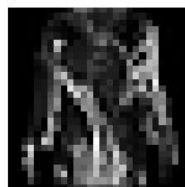
Sandal



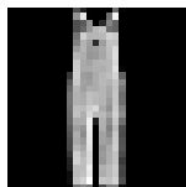
T-shirt/top



Shirt



Trouser

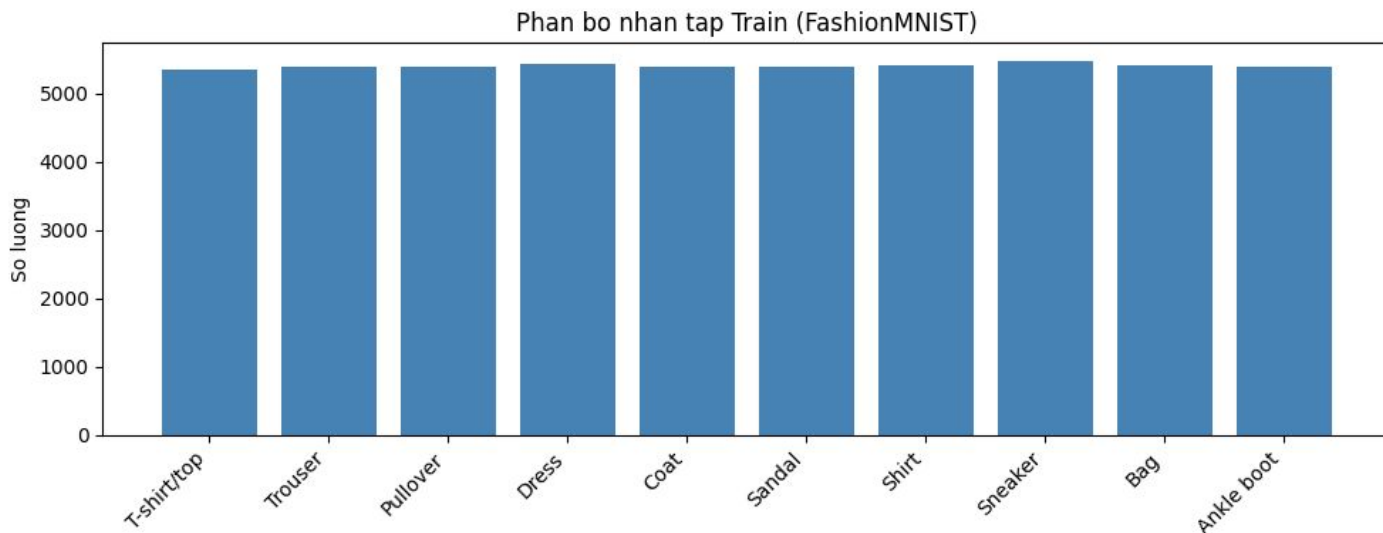


Ankle boot



Phân tích dữ liệu (EDA)

- **Phân phối lớp:** 10 lớp. Mỗi lớp gồm 5.000 ảnh/lớp.
- **Tỷ lệ max/min:** 1.00 => Dataset cân bằng.
- **Kích thước ảnh:** 28 x 28.
- **Số channel:** 01 do ảnh là grayscale.

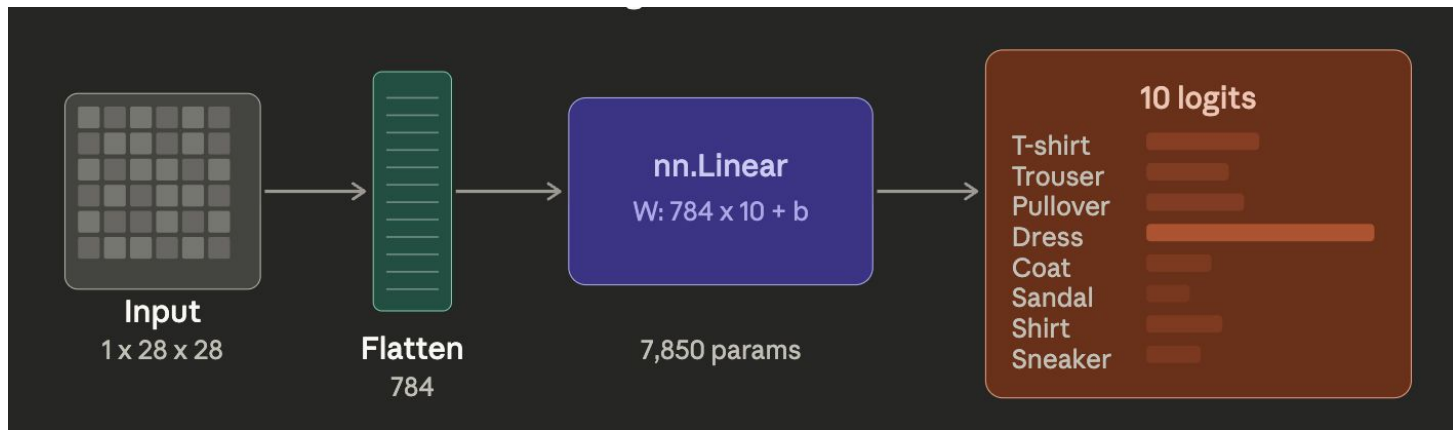


02

Xây dựng các mô hình phân loại

Softmax Regression trên Fashion-MNIST

- Input: $1 \times 28 \times 28 = 784$ pixel values | Output: 10 class
- Linear(784, 10): $784 \times 10 + 10$ (bias) = 7840 + 10 = 7850
- Model này coi mỗi ảnh như 1 vector 784 chiều rồi nhân với ma trận trọng số W (784×10) để tạo ra 10 logits tương tự 10 loại quần áo. Bản chất của nó là **multinomial logistic regression**.
- Hạn chế: mỗi pixel là 1 feature độc lập $\rightarrow$ Không hiểu spatial structure (cạnh, texture, hình dáng)
 $\rightarrow$ Accuracy bị giới hạn



MLP (Multi-Layer Perceptron)

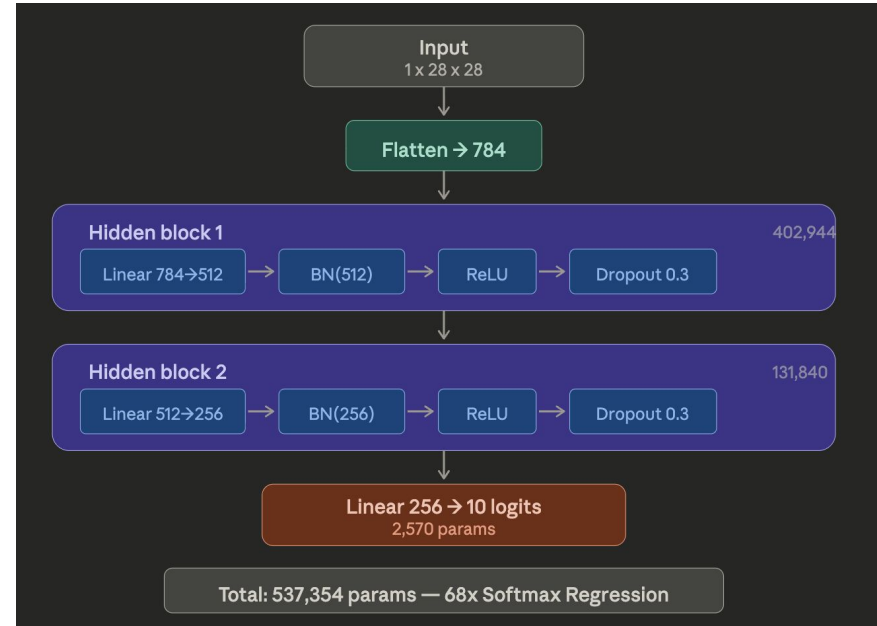
So với Softmax Regression chỉ có 1 lớp Linear, MLP thêm **hidden layers** với pattern lặp lại: **Linear** → **BatchNorm** → **ReLU** → **Dropout**.

Code dùng `hidden_dims=[512, 256]`, tức 2 hidden layers:

- **Layer 1:** Linear(784→512) → BN(512) → ReLU → Dropout(0.3)
- **Layer 2:** Linear(512→256) → BN(256) → ReLU → Dropout(0.3)
- **Output:** Linear(256→10)

Mỗi block gồm: **Linear** học features, **BatchNorm** ổn định training, **ReLU** thêm phi tuyến, **Dropout** chống overfitting. Tổng **537,354 params** — gấp ~68 lần Softmax Regression.

- **Hạn chế:** MLP vẫn flatten ảnh thành vector → vẫn mất spatial information



CNN (Convolutional Neural Network)

So với MLP flatten mất hết spatial info, CNN giữ nguyên cấu trúc 2D của ảnh qua 3 conv blocks, mỗi block theo pattern

Conv → **BN** → **ReLU** → **Pool**.

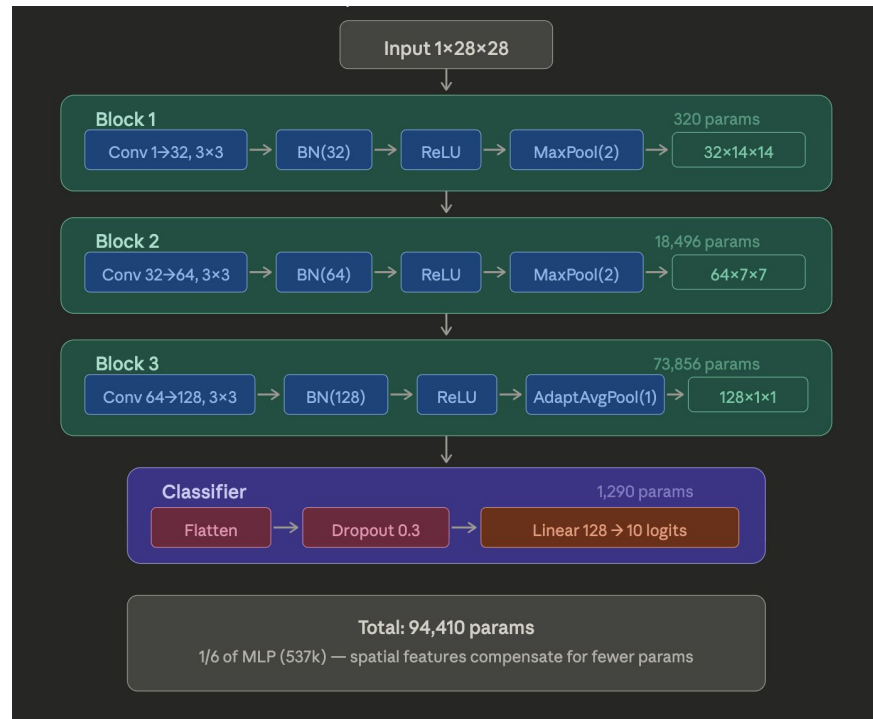
Feature extractor — 3 blocks tăng dần channels, giảm dần spatial size:

- **Block 1:** Conv(1→32, 3×3) → BN → ReLU → MaxPool(2) → output: **32×14×14**
- **Block 2:** Conv(32→64, 3×3) → BN → ReLU → MaxPool(2) → output: **64×7×7**
- **Block 3:** Conv(64→128, 3×3) → BN → ReLU → AdaptiveAvgPool(1) → output: **128×1×1**

Classifier: Flatten(128) → Dropout(0.3) → Linear(128→10)

Feature maps **tăng channels, giảm spatial**: $1 \times 28 \times 28 \rightarrow 32 \times 14 \times 14 \rightarrow 64 \times 7 \times 7 \rightarrow 128 \times 1 \times 1$ — mỗi block extract features trừu tượng hơn (edges → textures → patterns)

Tổng **94,410 params** — chỉ bằng 1/6 MLP nhưng hiệu quả hơn vì convolution lấy được cạnh, textures, khối.



Python built-in ViT (Vision Transformer)

Pipeline:

1. **Patch Embedding:** `Unfold` chia ảnh 28×28 thành 16 patches (mỗi patch $7 \times 7 = 49$ pixels), rồi `Linear(49→128)` project mỗi patch thành vector 128 chiều
2. **[CLS] token:** Prepend 1 learnable token đặc biệt → sequence length = 17
3. **Positional Embedding:** Cộng thêm learnable position cho mỗi token (vi Transformer không biết thứ tự)
4. **Transformer Encoder:** 4 layers, mỗi layer gồm Multi-Head Self-Attention (4 heads) + FFN ($128 \rightarrow 256 \rightarrow 128$), dùng GELU, `norm_first=True` (Pre-LN)
5. **Classification Head:** Lấy [CLS] token output → `LayerNorm` → `Linear(128→10)`.

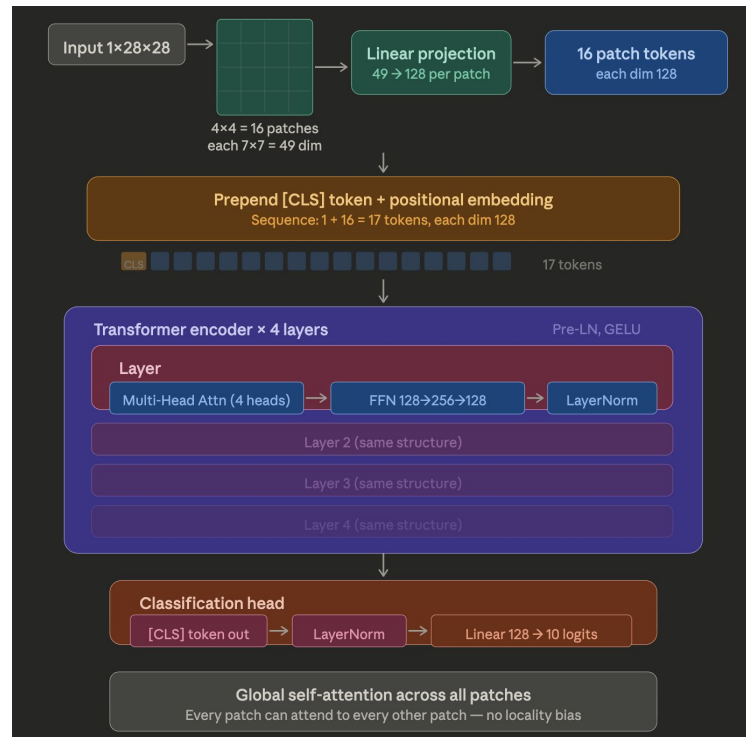
Chia ảnh thành patches thay vì dùng convolution — mỗi patch 7×7 trở thành 1 token, giống word trong NLP

[CLS] token là token đặc biệt tổng hợp thông tin từ tất cả patches qua self-attention, cuối cùng dùng để classify

Self-attention cho phép mỗi patch "nhìn" mọi patch khác — khác CNN chỉ nhìn local neighborhood

Pre-LN (`norm_first=True`) giúp training ổn định hơn Post-LN truyền thống

So sánh với CNN: ViT không có inductive bias về locality/translation invariance → cần nhiều data hơn để train tốt, nhưng scale tốt hơn trên large datasets

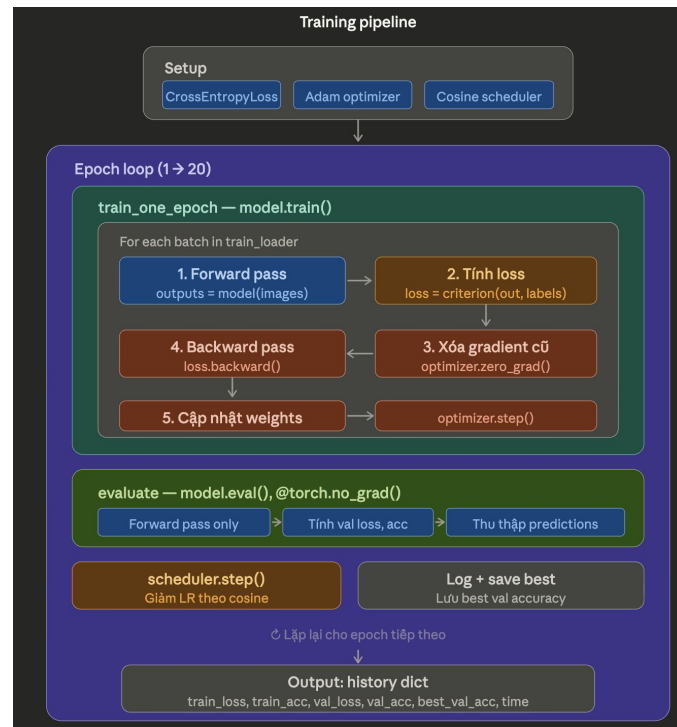


03

Huấn luyện, đánh giá và so sánh

Huấn luyện 4 mô hình với Pytorch

- Training loop: Mỗi epoch gồm 2 phase — train (cập nhật weights) và evaluate (đánh giá trên validation set)
- Forward → Loss → Zero Grad → Backward → Step: 5 bước chuẩn của PyTorch training, lặp lại cho mỗi batch
- `model.train()` vs `model.eval()`: Quan trọng vì BatchNorm và Dropout hoạt động khác nhau giữa train và inference
- `@torch.no_grad()`: Tắt tính gradient khi evaluate → tiết kiệm memory và tăng tốc
- Optimizer: Adam với weight decay $1e-4$ (L2 regularization) — adaptive learning rate cho từng parameter
- Scheduler: CosineAnnealingLR — giảm learning rate dần theo hình cosine từ LR ban đầu xuống ~ 0 , giúp fine-tune weights ở cuối training
- Best model tracking: Lưu best val accuracy để chọn checkpoint tốt nhất, tránh chọn model đã overfit



Kết quả huấn luyện 4 mô hình

- Softmax**: Bão hòa ngay epoch 5, chỉ cải thiện 3% trong 19 epochs còn lại → underfit do capacity quá thấp
 - MLP**: Hội tụ nhanh nhất (epoch 1 đã 85.4%) nhưng overfit gap lớn nhất (2.23%) → Dropout 0.3 chưa đủ kiểm chế
 - CNN**: Generalize tốt nhất — gap chỉ 0.31%, val loss thấp nhất (0.2354) và vẫn đang giảm ở epoch 20 → có thể cải thiện thêm nếu train lâu hơn
 - ViT**: Hội tụ chậm nhất, tốn thời gian nhất (376s vs ~275s) nhưng gap nhỏ (0.63%) → model có tiềm năng nếu tăng epochs và data
- => **Kết luận**: CNN cho training behavior lý tưởng nhất — stable convergence, minimal overfitting, val loss vẫn đang giảm

```
=====
[1/4] Huan luyen Softmax Regression
=====
```

```
Epoch [ 1/20] Train Loss: 0.7060 | Train Acc: 0.7615 | Val Loss: 0.6080 | Val Acc: 0.7907
Epoch [ 5/20] Train Loss: 0.5082 | Train Acc: 0.8274 | Val Loss: 0.5343 | Val Acc: 0.8167
Epoch [10/20] Train Loss: 0.4841 | Train Acc: 0.8349 | Val Loss: 0.5365 | Val Acc: 0.8157
Epoch [15/20] Train Loss: 0.4720 | Train Acc: 0.8405 | Val Loss: 0.5142 | Val Acc: 0.8235
Epoch [20/20] Train Loss: 0.4673 | Train Acc: 0.8425 | Val Loss: 0.5086 | Val Acc: 0.8280
Done! Time: 270.4s | Best Val Acc: 0.8288
```

```
=====
[3/4] Huan luyen CNN
=====
```

```
Epoch [ 1/20] Train Loss: 0.7397 | Train Acc: 0.7577 | Val Loss: 0.4817 | Val Acc: 0.8313
Epoch [ 5/20] Train Loss: 0.3460 | Train Acc: 0.8774 | Val Loss: 0.3362 | Val Acc: 0.8798
Epoch [10/20] Train Loss: 0.2878 | Train Acc: 0.8980 | Val Loss: 0.2814 | Val Acc: 0.9003
Epoch [15/20] Train Loss: 0.2475 | Train Acc: 0.9129 | Val Loss: 0.2597 | Val Acc: 0.9070
Epoch [20/20] Train Loss: 0.2314 | Train Acc: 0.9186 | Val Loss: 0.2354 | Val Acc: 0.9155
Done! Time: 294.0s | Best Val Acc: 0.9175
```

```
=====
[2/4] Huan luyen MLP
=====
```

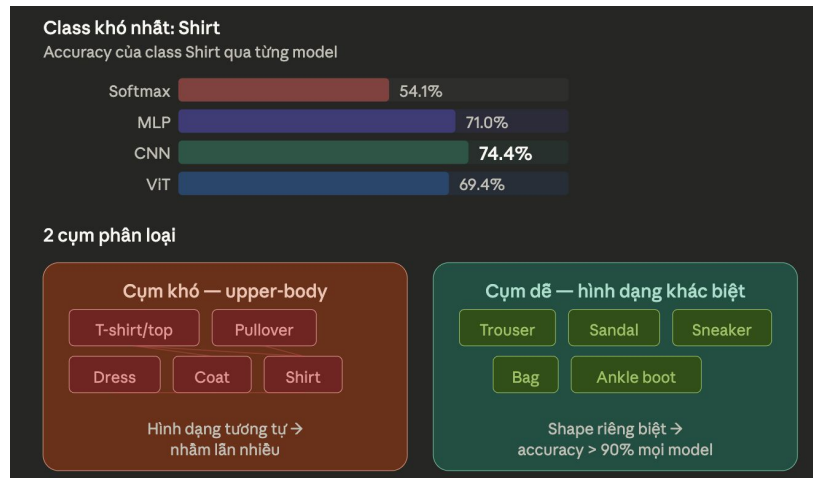
```
Epoch [ 1/20] Train Loss: 0.5270 | Train Acc: 0.8131 | Val Loss: 0.3964 | Val Acc: 0.8545
Epoch [ 5/20] Train Loss: 0.3371 | Train Acc: 0.8754 | Val Loss: 0.3361 | Val Acc: 0.8778
Epoch [10/20] Train Loss: 0.2790 | Train Acc: 0.8964 | Val Loss: 0.3021 | Val Acc: 0.8887
Epoch [15/20] Train Loss: 0.2250 | Train Acc: 0.9164 | Val Loss: 0.2787 | Val Acc: 0.8985
Epoch [20/20] Train Loss: 0.2006 | Train Acc: 0.9253 | Val Loss: 0.2667 | Val Acc: 0.9030
Done! Time: 275.0s | Best Val Acc: 0.9030
```

```
=====
[4/4] Huan luyen ViT (built-in TransformerEncoder)
=====
```

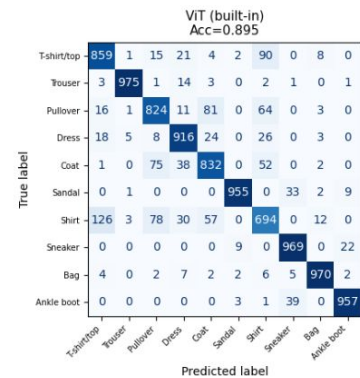
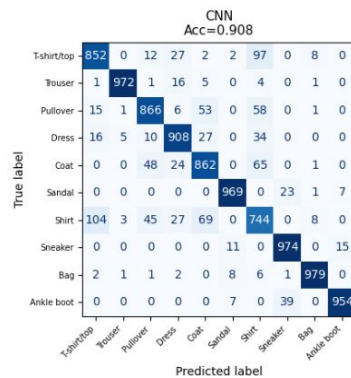
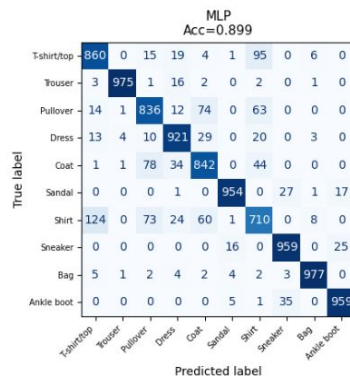
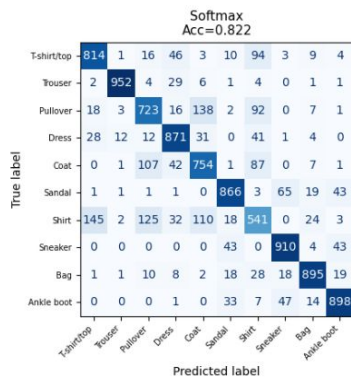
```
/tmp/ipykernel_1977/196319873.py:39: UserWarning: enable_nested_tensor is True, but self.use_
self.transformer = nn.TransformerEncoder(
Epoch [ 1/20] Train Loss: 0.7162 | Train Acc: 0.7388 | Val Loss: 0.4978 | Val Acc: 0.8207
Epoch [ 5/20] Train Loss: 0.3839 | Train Acc: 0.8581 | Val Loss: 0.3782 | Val Acc: 0.8638
Epoch [10/20] Train Loss: 0.3212 | Train Acc: 0.8814 | Val Loss: 0.3190 | Val Acc: 0.8882
Epoch [15/20] Train Loss: 0.2735 | Train Acc: 0.8973 | Val Loss: 0.2953 | Val Acc: 0.8907
Epoch [20/20] Train Loss: 0.2529 | Train Acc: 0.9056 | Val Loss: 0.2809 | Val Acc: 0.8993
Done! Time: 376.0s | Best Val Acc: 0.8993
```

So sánh 4 mô hình

- Shirt là class khó nhất ở mọi model: Softmax 54.1% → MLP 71.0% → CNN 74.4% (tốt nhất) → ViT 69.4%.
- Cụm upper-body (T-shirt, Pullover, Dress, Coat, Shirt) là nguồn lỗi chính — silhouette gần giống nhau, khó phân biệt bằng hình dạng tổng quát.
- Cụm easy (Trouser, Sandal, Sneaker, Bag, Ankle boot) đạt >90% accuracy ở mọi model, kể cả Softmax — shape hoàn toàn khác biệt.
- CNN vượt trội ở cụm khó nhờ convolution capture được texture và edge details (đường may, cổ áo, pattern vải) — đây chính là lợi thế của inductive bias về locality.
- ViT kém CNN ở Shirt (69.4% vs 74.4%) — trên ảnh 28×28, self-attention thiếu chi tiết pixel để phân biệt các loại áo



Confusion Matrix — 4 mô hình



Tổng hợp so sánh

Mo hình	Test Acc	Test F1	Params	Time (s)
Softmax	0.8224	0.8213	7,850	270.4
MLP	0.8993	0.8990	537,354	275.0
CNN	0.9080	0.9081	94,410	294.0
ViT (built-in)	0.8951	0.8947	540,170	376.0

- **Softmax Regression** (82.24%): Baseline đơn giản nhất, chỉ có phép biến đổi tuyến tính, không trích xuất features → accuracy thấp nhất
- **MLP** (89.93%): Cải thiện nhờ hidden layers phi tuyến, nhưng flatten ảnh thành vector → mất hoàn toàn cấu trúc không gian, tốn nhiều params nhất (537k)
- **CNN** (90.80%): Accuracy cao nhất, params ít nhất (94k) — nhờ khai thác locality và weight sharing, hiệu quả nhất trên dataset nhỏ
- **ViT** (89.51%): Global self-attention cho phép mỗi patch nhìn toàn bộ ảnh, competitive với CNN nhưng cần nhiều params (540k) và data lớn hơn để vượt CNN

=> **Kết luận:** Trên Fashion-MNIST (28×28, 60k samples), CNN là lựa chọn tối ưu nhất về cả accuracy lẫn hiệu quả tham số. ViT phát huy sức mạnh thực sự trên large-scale datasets (ImageNet, JFT-300M)

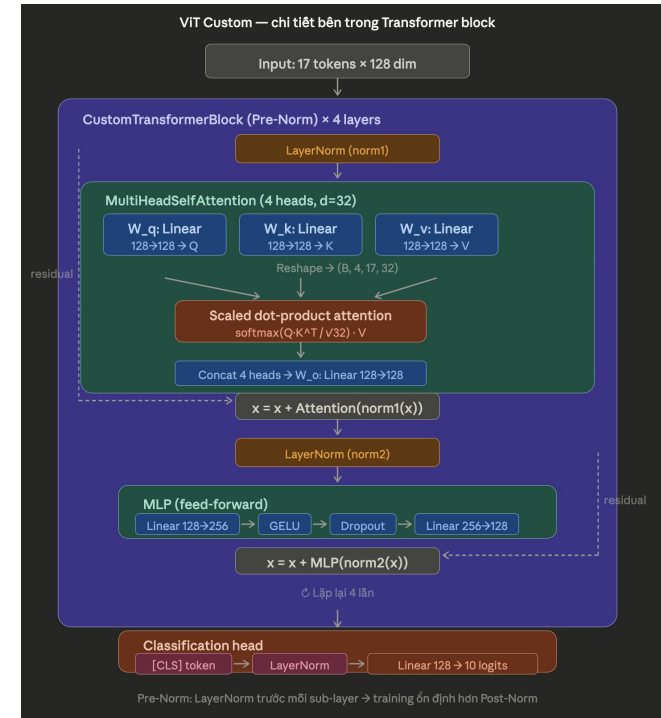
04

Tự hiện thực

TransformerEncoder & ViT

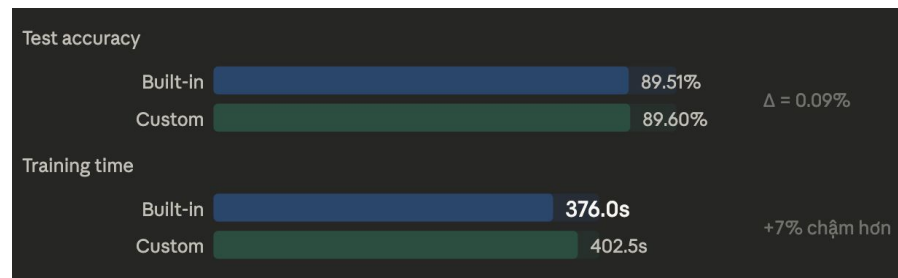
Mô hình ViT Custom

- Tự hiện thực Attention: Q, K, V được tạo bởi 3 linear layers riêng biệt, reshape thành 4 heads (mỗi head 32 dim), tính $\text{softmax}(Q \cdot K^A T / \sqrt{32}) \cdot V$ bằng einsum.
- Scale factor $\sqrt{d}$: Chia cho $\sqrt{32} \approx 5.66$ để tránh dot product quá lớn khiến softmax bị saturate (gradient gần 0).
- Multi-head: 4 heads nhìn input từ 4 "góc nhìn" khác nhau, concat lại rồi project qua W_o — tăng khả năng biểu diễn.
- Pre-Norm (norm trước): $x + \text{Attn}(\text{LN}(x))$ thay vì $\text{LN}(x + \text{Attn}(x))$ — gradient flow qua residual path không bị LayerNorm chặn → training ổn định hơn, đặc biệt khi stack nhiều layers.
- Residual connections: Cộng trực tiếp input vào output mỗi sub-layer → tránh vanishing gradient, cho phép stack sâu.
- GELU thay vì ReLU: Smooth activation, không "chết" ở vùng âm như ReLU — tiêu chuẩn trong Transformer.
- So với Built-in: Kết quả hoàn toàn tương đương — mục đích chính là hiểu rõ cơ chế bên trong thay vì dùng "hộp đen"

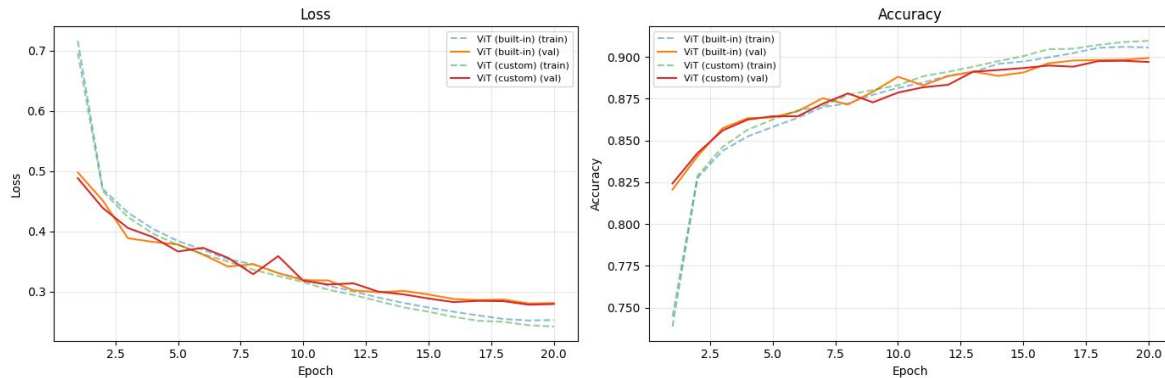


So sánh ViT built-in với ViT Custom

- Accuracy gần như giống hệt: Built-in 89.51% vs Custom 89.60% — Δ chỉ 0.09%, nằm trong khoảng dao động ngẫu nhiên.
- Built-in nhanh hơn 7%: 376s vs 402.5s — nhờ PyTorch dùng C++/CUDA optimized kernels và fused attention operations bên dưới.
- Custom chậm hơn nhưng minh bạch: Từng phép tính Q, K, V, scaled dot-product đều viết tường minh bằng Python → dễ debug, dễ hiểu, dễ modify.
- Cùng kiến trúc, cùng hyperparameters: patch_size=7, embed_dim=128, num_heads=4, num_layers=4, mlp_dim=256 → khác biệt chỉ do random initialization.
- Khi nào dùng gì: Production → dùng built-in (nhanh, tối ưu). Nghiên cứu/học tập → dùng custom (hiểu rõ cơ chế, dễ thay đổi attention mechanism).



So sánh ViT Built-in vs ViT Custom

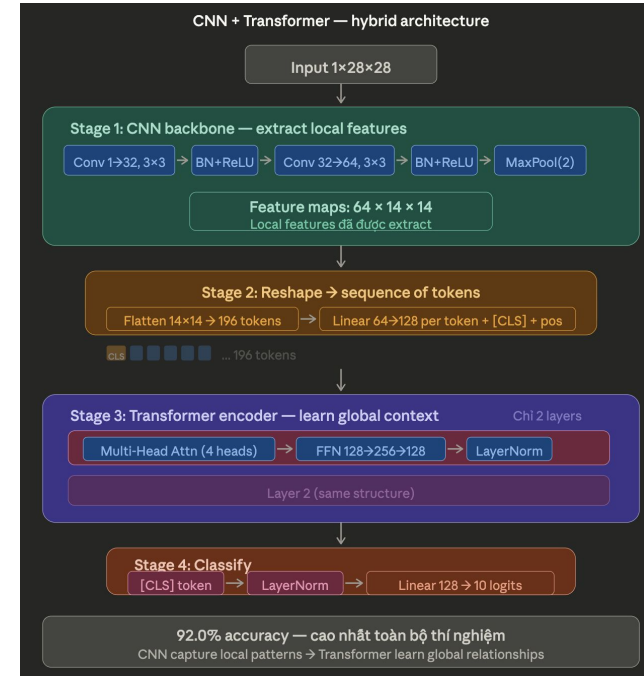


05

Các kiến trúc kết hợp và cách embed ảnh khác nhau

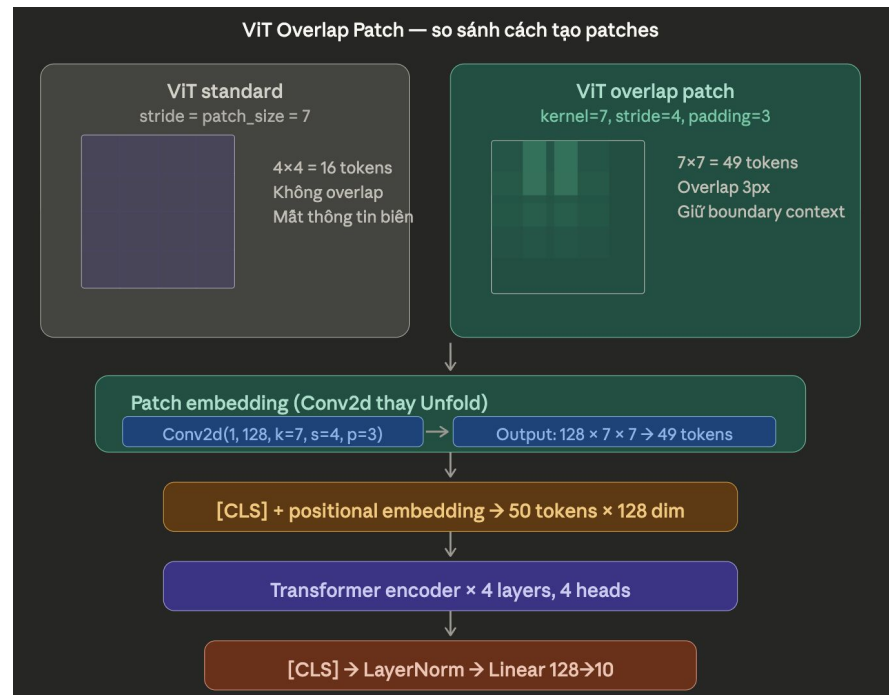
Mô hình CNN + Transformer

- **Ý tưởng cốt lõi:** Dùng CNN extract local features (edges, textures) trước, rồi đưa vào Transformer để học global relationships giữa các vùng ảnh
- **CNN backbone** (2 conv layers): Biến ảnh $1 \times 28 \times 28 \rightarrow$ feature maps $64 \times 14 \times 14$ — mỗi pixel trong feature map đã chứa thông tin local context
- **Reshape thành tokens:** Flatten $14 \times 14 = 196$ spatial positions, mỗi position project lên 128 dim $\rightarrow$ Transformer nhận 197 tokens (196 + [CLS])
- **Transformer chỉ cần 2 layers** (thay vì 4 ở ViT thuần) — vì input đã là features chất lượng cao từ CNN, không phải raw pixels
- **Tại sao hiệu quả nhất (92.0%):** CNN cung cấp inductive bias về locality $\rightarrow$ Transformer không cần học from scratch, chỉ cần tổng hợp global context từ local features
- **So sánh:** CNN thuần (90.8%) chỉ nhìn local $\rightarrow$ thiếu global. ViT thuần (89.5%) phải học cả local lẫn global từ raw pixels $\rightarrow$ không đủ data. Hybrid kết hợp cả hai $\rightarrow$ vượt trội
- **Bài học thiết kế:** Trên small datasets, hybrid CNN+Transformer là chiến lược tối ưu — đừng bỏ hết inductive bias như ViT thuần, mà tận dụng CNN làm "bộ tiền xử lý" cho Transformer



Overlapping Patch Embedding

- **Vấn đề của ViT standard:** `Unfold(stride=7)` chia ảnh thành 16 patches không chồng nhau → thông tin ở biên giữa 2 patches bị "cắt đôi", không patch nào nhìn đầy đủ
- **Giải pháp:** Dùng `Conv2d(kernel=7, stride=4, padding=3)` — mỗi patch vẫn nhìn vùng 7×7 nhưng chỉ dịch 4 pixels → overlap 3 pixels giữa patches liền kề
- **Nhiều tokens hơn:** $7 \times 7 = 49$ tokens (vs 16 tokens ViT standard) → Transformer có thông tin spatial chi tiết hơn gấp 3 lần
- **Conv2d = Unfold + Linear gộp thành 1 bước** — vừa tạo patches vừa project lên `embed_dim=128`, hiệu quả hơn
- **Kết quả:** 90.1% vs 89.5% — cải thiện 0.6% chỉ bằng cách thay đổi cách tạo patches, giữ nguyên toàn bộ Transformer
- **Trade-off:** Nhiều tokens hơn (49 vs 16) → attention $O(n^2)$ tốn hơn (377s vs 376s, chênh lệch nhỏ vì ảnh 28×28 nhỏ, trên ảnh lớn sẽ rõ hơn)
- **Insight:** Cách biểu diễn input quan trọng không kém kiến trúc model — overlap nhỏ nhưng ảnh hưởng lớn đến chất lượng



Spatial Token (HxW as token)

Ý tưởng thí nghiệm: Bỏ hoàn toàn patching, coi mỗi pixel là 1 token — kiểm tra xem Transformer có thể học từ raw pixels không

Pipeline: Ảnh (1,28,28) → flatten → 784 tokens (mỗi token dim=1) → Linear(1→128) → [CLS] + pos → Transformer 2 layers → classify

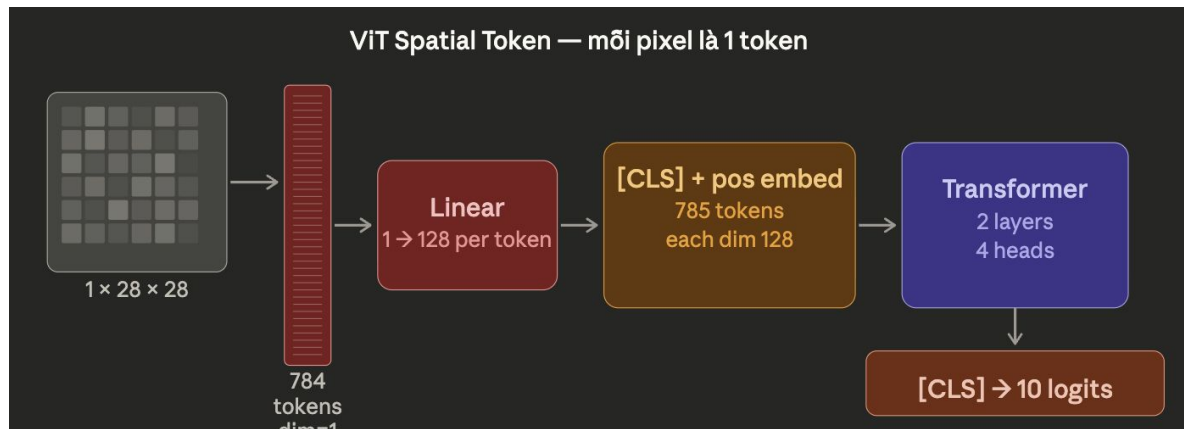
Kết quả tệ nhất: 84.5% accuracy, 3078s training — chậm gấp 8× và kém hơn cả Softmax Regression (82.2%) chỉ ~2%

Tại sao chậm: Self-attention $O(n^2)$ — với 784 tokens: $784^2 = 614,656$ attention scores mỗi layer, so với $16^2 = 256$ ở ViT standard (gấp 2,400×)

Tại sao kém: Mỗi token chỉ mang 1 giá trị pixel, không chứa thông tin về neighborhood → attention phải tự phát hiện "pixel 100 cạnh pixel 101" hoàn toàn từ positional embedding — quá khó với 2 layers

Chỉ dùng 2 layers (thay vì 4) vì sequence quá dài → giảm capacity, nhưng vẫn cực chậm

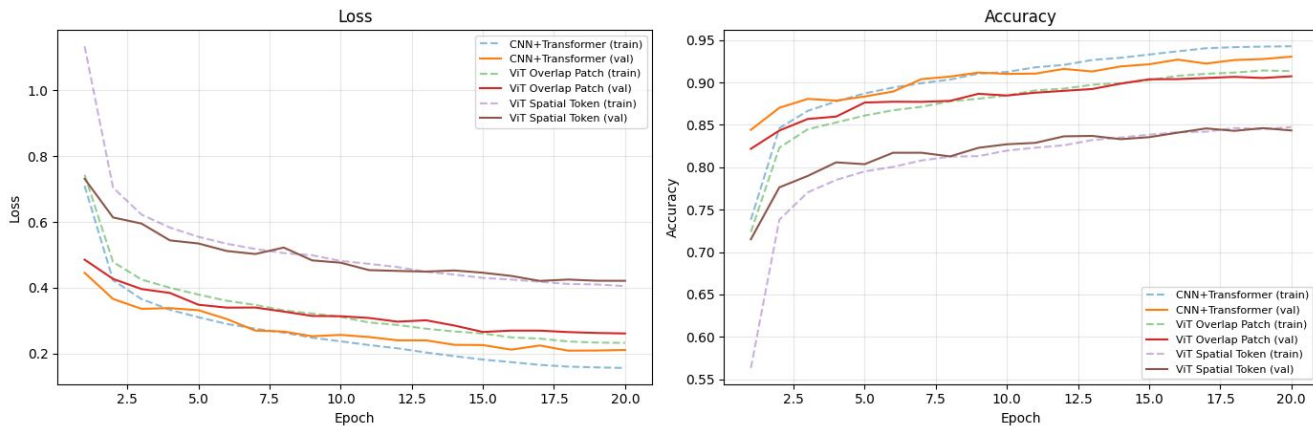
So sánh: ViT standard (16 tokens, 89.5%) > Overlap (49 tokens, 90.1%) >> Spatial (784 tokens, 84.5%) — nhiều tokens hơn không có nghĩa tốt hơn



So sánh 3 mô hình

- **CNN+Transformer hội tụ nhanh nhất:** Epoch 5 đã đạt ~89% val acc, cuối training lên 92% — CNN backbone cho "warm start", Transformer refine global context.
 - **CNN+Transformer train loss thấp nhất** (~0.12) nhưng val loss ~0.22 → overfitting nhẹ (~3% gap) — có thể cải thiện bằng tăng dropout hoặc data augmentation.
 - **ViT Overlap Patch hội tụ đều đặn:** Loss/accuracy curves mượt, gap train-val vừa phải — overlap patches cung cấp đủ local context để model học ổn định.
 - **ViT Spatial Token bão hòa sớm:** Val loss phẳng từ epoch 7, val acc đứng yên ~85% — 784 tokens × 2 layers không đủ capacity và quá nhiều noise.
 - **Spatial Token overfitting tệ nhất:** Train loss vẫn giảm nhưng val loss không theo → model ghi nhớ training data thay vì học features tổng quát.
- => **Kết luận:** Input representation quyết định training behavior — CNN features > overlapping patches > raw pixels. CNN+Transformer cho kết quả tốt nhất nhờ kết hợp inductive bias của CNN với global attention của Transformer

Phan 4: So sánh kiến trúc kết hợp

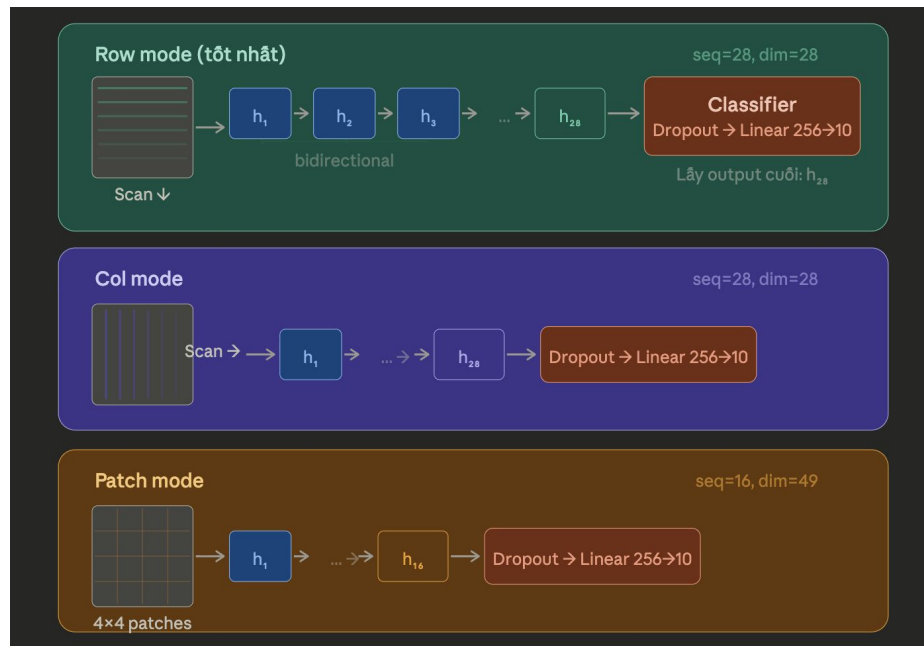


06

Mô hình phân loại dựa trên LSTM/GRU

RNN Image Classifier

- **Ý tưởng:** Biến ảnh 2D thành sequence 1D rồi xử lý bằng RNN — 3 cách: row (scan ngang), col (scan dọc), patch (chia miếng).
- **Bidirectional:** Mỗi RNN layer chạy 2 chiều (thuận + nghịch) → output dim = $128 \times 2 = 256$, capture context cả 2 hướng.
- **Lấy output cuối** (`output[:, -1, :]`): Hidden state ở timestep cuối chứa thông tin tổng hợp toàn bộ sequence → đưa vào classifier.
- **Row mode tốt nhất** (LSTM 90.28%, GRU 90.86%): Mỗi hàng pixel chứa thông tin chiều rộng quần áo (vai, eo, gấu) — đặc trưng phân biệt giữa các class.
- **Col kém hơn Row 1.15%:** Thông tin vertical (chiều dọc) ít đặc trưng hơn horizontal cho Fashion-MNIST.
- **Patch kém nhất** (88.46%): Chia thành patches rời rạc → RNN phải nhớ context qua nhiều timesteps giữa các patches không liền kề, khó hơn row scan liên tục.
- **GRU hiệu quả hơn LSTM:** Ít gates (2 vs 3), ít params (420k vs 559k), nhanh hơn, accuracy tương đương hoặc nhỉnh hơn — trên sequence ngắn (28 timesteps) không cần forget gate phức tạp của LSTM.
- **So với CNN:** RNN row (90.86%) gần bằng CNN (90.80%) — khá sát nhau, cho thấy sequential patterns trong ảnh quần áo cũng rất informative.



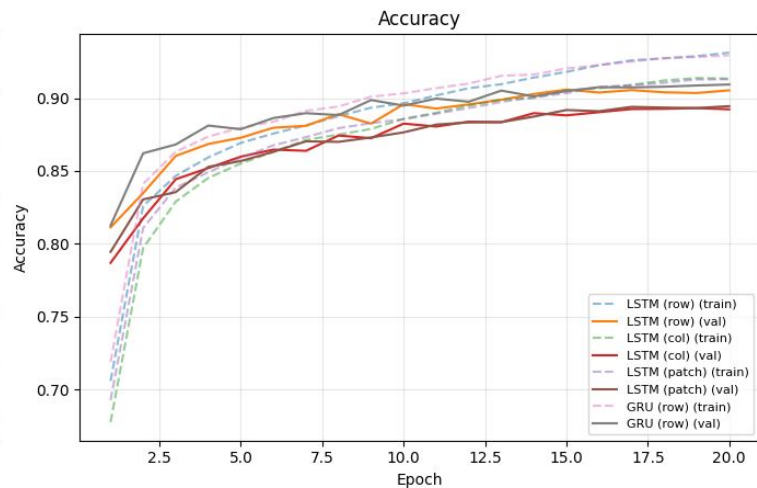
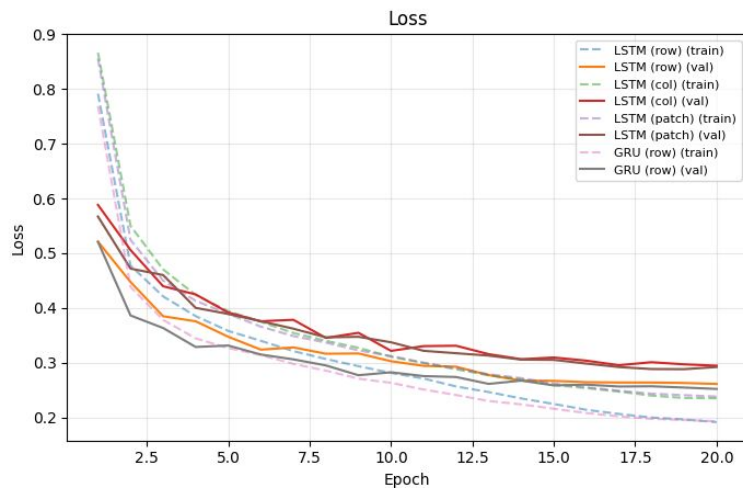
So sánh kết quả

Mode	Accuracy	Params	Insight
LSTM row	90.28%	559,626	Horizontal info tốt nhất
GRU row	90.86%	420,362	Ít params, nhanh hơn
LSTM col	89.13%	559,626	Vertical kém hơn
LSTM patch	88.46%	581,130	Mất spatial continuity

So sánh LSTM/GRU với các cách biểu diễn chuỗi

- Row-based tốt nhất (LSTM 90.28%, GRU 90.86%) — scan từ trên xuống tự nhiên nhất cho quần áo, mỗi hàng chứa thông tin silhouette đặc trưng.
 - Col kém hơn Row 1.15% (89.13% vs 90.28%) — thông tin horizontal (chiều rộng vai, eo) phân biệt class tốt hơn vertical.
 - Patch kém nhất (88.46%) — chia thành patches phá vỡ spatial continuity, RNN khó liên kết thông tin rời rạc giữa các patches.
 - GRU $\approx$ LSTM nhưng hiệu quả hơn — ít gates hơn (2 vs 3), ít params hơn (420k vs 559k), nhanh hơn, accuracy tương đương hoặc nhỉnh hơn trên sequence ngắn.
 - Tất cả RNN đều overfit nhẹ — gap train-val $\sim 2\text{-}3\%$, train loss tiếp tục giảm trong khi val loss bắt đầu phẳng sau epoch 15.
- = >**Kết luận**: Trên Fashion-MNIST, cách biểu diễn sequence quan trọng hơn kiến trúc RNN — row scan là tối ưu cho dạng ảnh quần áo

Phan 5: So sánh LSTM/GRU với các cách biểu diễn chuỗi



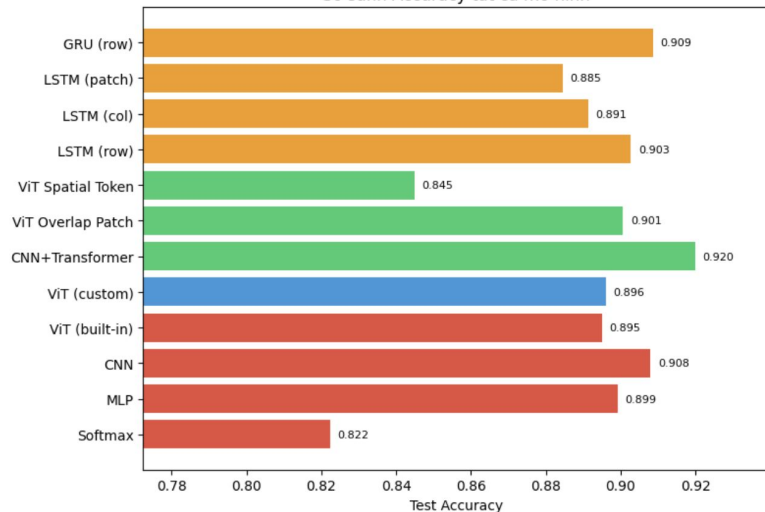
07

Tổng hợp kết quả

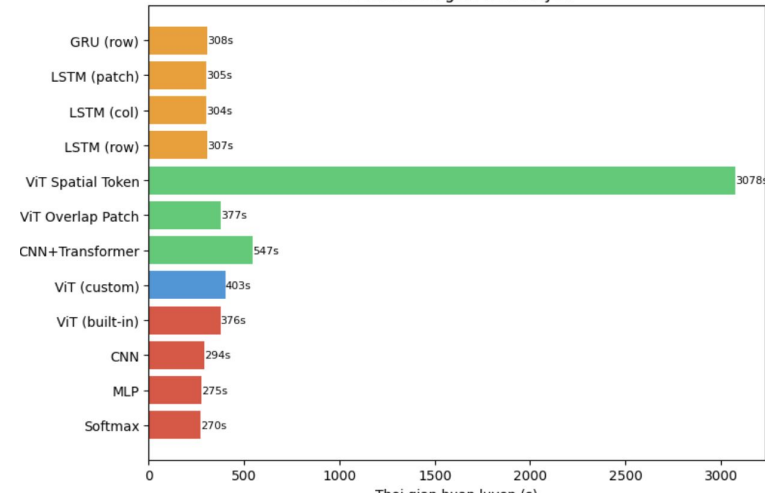
Tổng hợp kết quả

- **CNN+Transformer đạt accuracy cao nhất (92.0%)** — CNN extract local features, Transformer học global context → best of both worlds
- **GRU/LSTM row competitive (~90-91%)** — xử lý ảnh theo hàng pixel capture được sequential shape patterns, GRU row thậm chí vượt CNN thuần
- **LSTM row > LSTM col (90.3% vs 89.1%)** — thông tin horizontal (chiều rộng quần áo) quan trọng hơn vertical trên Fashion-MNIST
- **ViT Spatial Token kém nhất nhóm nâng cao (84.5%, 3078s)** — tokenize từng pixel tạo sequence quá dài (784 tokens), $O(n^2)$ attention cực chậm mà thiếu local context
- **ViT Overlap Patch (90.1%) vượt ViT standard (89.5%)** — overlapping patches giữ được thông tin biên giữa các patches
- **Hiệu quả tổng thể:** CNN (90.8%, 294s) vẫn là lựa chọn tối ưu nhất nếu xét accuracy/time/params. CNN+Transformer (92.0%, 547s) đáng trade-off nếu cần accuracy cao nhất
- **Kết luận:** Trên small-scale dataset, inductive bias (locality của CNN, sequential của RNN) giúp model học hiệu quả hơn. Transformer thuần cần scale lên để phát huy sức mạnh

So sánh Accuracy tất cả mô hình



So sánh thời gian huấn luyện





Thank you!